

系统架构设计入门

一、架构设计原则

系统架构是软件系统的骨架，决定了系统的可维护性、可扩展性和可靠性。SOLID原则是面向对象设计的五大基石：单一职责原则（一个类只负责一件事）、开闭原则（对扩展开放、对修改关闭）、里氏替换原则、接口隔离原则和依赖倒置原则（依赖抽象而非具体实现）。KISS告诫避免过度设计——简单的方案通常更可靠。YAGNI提醒不要为将来可能不会出现的需求提前编码。DRY强调消除重复代码和重复逻辑。

二、微服务与单体架构

单体架构将应用的所有功能打包在一个进程中，开发和部署简单，但随着规模增长代码库膨胀、部署耦合、技术栈锁定等问题逐渐暴露。微服务架构将大型应用拆分为一组小型独立服务，每个服务围绕特定业务能力构建、独立部署和伸缩、可使用不同技术栈。微服务间的通信分为同步（HTTP REST、gRPC）和异步（消息队列、事件总线）两种方式。服务发现解决微服务实例动态变化时的寻址问题——Consul、Etcd、Nacos是常用方案。API网关是微服务对外的统一入口，负责路由、限流、认证、日志等横切关注点。

三、数据库架构与优化

数据库是大多数系统的性能瓶颈所在。读写分离将读请求路由到从库、写请求发送到主库，通过MySQL主从复制实现。分库分表将数据按规则分散到多个数据库实例——水平分表按行拆分（如按用户ID哈希），垂直分表按列拆分（将大字段分离到扩展表）。缓存是提升读性能的最有效手段——Redis和Memcached是主流选择。缓存策略包括Cache-Aside（先查缓存、未命中则查数据库后回填）、Write-Through（同步更新缓存和数据库）。缓存穿透、缓存击穿和缓存雪崩是三个需要防范的常见问题——布隆过滤器可防止查询不存在的数据穿透到数据库层。数据库索引（Index）优化是另一个关键方向——MySQL InnoDB的B+树索引性能受索引选择性、回表次数、索引覆盖等因素影响。使用慢查询日志结合EXPLAIN分析SQL执行计划，是定位数据库性能瓶颈的标准方法。复合索引遵循最左前缀原则。

四、高可用与负载均衡

高可用系统通过冗余设计消除单点故障。负载均衡器将流量分发到多个服务实例——Nginx和HAProxy是常用方案。负载均衡算法包括轮询、最少连接、IP哈希、加权轮询等。健康检查定期探测后端服务状态，自动摘除故障节点。故障转移在主节点故障时自动切换到备用节点。多活架构比主备更优，所有节点同时提供服务。熔断器模式防止级联故障——当某个服务的失败率超过阈值时暂时停止向它发送请求。限流保护系统免于过载——令牌桶算法和滑动窗口是最常用的限流算法。Python的异步框架如asyncio和FastAPI天然适合构建高并发的网关和中间件。

五、安全架构基础

安全是系统架构不可忽视的维度。身份认证确认用户是谁——JWT是无状态的认证方案，OAuth 2.0是授权委托的标准协议。授权确定用户能做什么——RBAC（基于角色的访问控制）是最常见的授权模型。HTTPS/TLS加密保护数据在网络传输中的安全。SQL注入是最古老的Web安全漏洞之一，使用参数化查询或ORM框架可有效防范。XSS攻击通过在页面中注入恶意脚本窃取用户信息。CSRF攻击利用用户的已登录状态发起恶意请求。日志审计记录所有关键操作，用于事后追溯和合规检查。Python中可以使用bcrypt库进行密码哈希，使用secrets模块生成安全的随机令牌。